

Operating System Lab Manual

Course: Operating System and System Programming Lab

Course Code : CSE 3204

Duration: 14 Weeks

Week 1

Topic:

Introduction to Linux & Ubuntu Environment

Objective:

- Understand the role of an operating system and where Linux fits among common OS families.
- Get comfortable navigating Ubuntu through both the graphical interface and the terminal.
- Learn the basic anatomy of the Linux shell and how commands are structured.

Background / Theory:

An operating system manages hardware resources (CPU, memory, storage, I/O) and provides a platform for running applications. Linux is a Unix-like, open-source kernel; Ubuntu is a popular Linux distribution built around it, paired with the GNOME desktop and the Bash shell for command-line interaction.

The shell is a command interpreter: it reads a command line, parses the command name and its arguments/options (flags beginning with - or --), and executes the corresponding program. Most Linux commands follow the pattern: command [options] [arguments].

Activities:

- Log in to Ubuntu and explore the desktop environment, file manager, and settings.
- Open a terminal and try: **pwd**, **whoami**, **hostname**, **uname -a**, **date**, **cal**, **uptime**.
- Use **man <command>** and **command --help** to read documentation for at least three commands.
- Explore command history using the up/down arrows and the history command.

- Use clear and Ctrl+L to clear the terminal screen.

Lab Tasks:

1. Open a terminal and print your current working directory, username, and system information (kernel version, OS name) using appropriate commands.
2. Display today's date and a calendar for the current month.
3. Use the man pages to look up the ls and cp commands; note two options for each you had not seen before.
4. Run at least 10 different commands, then use history to list your last 10 commands and record them in your lab report.
5. Write a short paragraph explaining the difference between a kernel, a shell, and a terminal emulator.

Viva / Review Questions:

- What is the difference between an operating system and a kernel?
 - Why is Linux described as a multi-user, multitasking operating system?
 - What does the shell prompt (\$ vs #) tell you about the current user?
-

Week 2

Topic:

File and Directory Management

Objective:

- Create, organize, copy, move, and delete files and directories from the command line.
- Understand the Linux filesystem hierarchy and absolute vs. relative paths.
- Search for files using pattern-based tools.

Background / Theory:

Linux organizes files in a single hierarchical tree rooted at /. Every file or directory is reached by an absolute path (starting from /) or a relative path (starting from the current directory, referenced by . and ..). Common top-level directories include /home, /etc, /var, /bin, and /usr.

Activities:

- `mkdir dirname / mkdir -p a/b/c` — create directories, including nested ones in one step('a' is parent directory, 'b' is inside 'a' and 'c' is inside 'b')

- `ls`, `ls -l`, `ls -a`, `ls -lh` — list directory contents with detail, hidden files, and human-readable sizes.
- `cp file1 file2` and `cp -r dir1 dir2` — copy files and directories recursively.
- `mv old new` — move or rename files and directories.
- `rm file`, `rm -r dir`, `rmdir emptydir` — delete files, non-empty directories, and empty directories.
- `find . -name "*.txt"`, `find . -type d`, `find . -mtime -1` — search by name, type, and modification time.
- `touch filename` — create an empty file or update its timestamp.
- `tree` (if installed) — visualize a directory structure.

Lab Tasks:

6. Create a directory structure: `labwork/docs`, `labwork/scripts`, `labwork/data` (in one `mkdir -p` command).
7. Create three empty files inside `labwork/docs` using `touch`, then list them with `ls -l`.
8. Copy one file from `labwork/docs` into `labwork/data`, then rename the copy using `mv`.
9. Use `find` to locate all `.txt` files under `labwork` and all files modified in the last day.
10. Delete the `labwork/scripts` directory (empty) with `rmdir` and the whole `labwork` tree with `rm -r`, and explain the difference between the two commands in your report.

Viva / Review Questions:

- What is the difference between an absolute path and a relative path?
- What happens if you run `rm -r` on a directory versus `rmdir`?
- How would you find all files larger than 1MB in your home directory?

Week 3

Topic:

File Viewing, Editing & Permissions

Objective:

- View and edit text files directly from the terminal.
- Understand the Linux file permission model and modify permissions and ownership.

- Write and execute a basic shell script.

Background / Theory:

Every file has an owner, a group, and permissions for three classes: user (owner), group, and others. Each class can have read (r), write (w), and execute (x) permission, shown as a 10-character string (e.g. -rwxr-xr--). Permissions can be set symbolically (chmod u+x file) or numerically (chmod 754 file, where each digit is the sum of r=4, w=2, x=1 for owner/group/others).

A shell script is a text file of commands executed by an interpreter such as Bash. It supports variables, conditionals (if/then/else), loops (for, while), and reads input with read.

Activities:

- cat file, less file, head -n 5 file, tail -n 5 file — view file contents in different ways.
- nano file or vi file — edit a text file in the terminal.
- chmod 755 script.sh and chmod u+x script.sh — change permissions numerically and symbolically.
- chown user:group file and ls -l to inspect ownership and permission strings.
- Write and run a simple Bash script using #!/bin/bash, variables, and an if statement.

Sample Shell Script (greet.sh):

```
#!/bin/bash
# greet.sh - simple shell script demonstrating variables and a conditional
echo "Enter your name:"
read name
if [ -z "$name" ]; then
    echo "No name entered!"
else
    echo "Hello, $name! Today is $(date '+%A, %d %B %Y')."
fi
```

Lab Tasks:

11. Create a text file with a few lines and view it using cat, less, head -n 2, and tail -n 2. Note the difference in output.

12. Edit the file using nano to add a new line, then verify the change with cat.
13. Create the greet.sh script above, make it executable using chmod, and run it with ./greet.sh.
14. Change the file's permissions so that only the owner can read and write it (no execute), and verify with ls -l.
15. Explain in your report what each character in the permission string -rwxr-xr-- means.

Viva / Review Questions:

- What is the numeric equivalent of the permission string rwxr-xr-x?
 - Why must a shell script be given execute permission before it can be run with ./script.sh?
 - What is the difference between chmod and chown?
-

Week 4

Topic:

Lab Assessment

Objective:

- Consolidate and demonstrate the Linux fundamentals covered in Weeks 1-3.
- Complete a graded practical assessment individually under supervision.

Background / Theory:

This session is a checkpoint assessment covering Linux navigation, file and directory management, permissions, and basic shell scripting from the previous three weeks. No new topics are introduced; instead, students apply the earlier material to a fresh set of problems.

Activities:

- Review terminal navigation commands (pwd, cd, ls) and file operations (mkdir, cp, mv, rm, find).
- Review permission commands (chmod, chown) and basic shell scripting constructs (variables, if, loops).
- Attempt a timed practical test combining these topics, e.g. creating a directory structure, setting specific permissions, and writing a short script that processes input.

Lab Tasks:

16. Given a small folder structure, reproduce it using only command-line instructions (no GUI).
17. Set specific, instructor-given permissions on two files using both symbolic and numeric notation.
18. Write a short shell script that accepts a filename as input and reports whether the file exists and is readable.
19. Submit your terminal command history and script file as evidence of work completed during the session.

Viva / Review Questions:

- Be prepared to explain, live, any command you used during the assessment.
 - Be ready to modify your script on the spot if asked to handle an edge case (e.g. missing input).
-

Week 5

Topic:

Process Management

Objective:

- Understand what a process is and how the OS tracks running processes.
- Monitor, control, and terminate processes from the command line.
- Understand process creation in Linux via the `fork()` system call.

Background / Theory:

A process is a program in execution, with its own memory space, program counter, and set of resources. Linux assigns each process a unique Process ID (PID) and a parent PID (PPID). Processes can run in the foreground (blocking the terminal) or background (appending `&` to the command).

`fork()` is a system call that creates a new process (the child) by duplicating the calling process (the parent). It returns 0 in the child, the child's PID in the parent, and a negative value on failure. This is the fundamental mechanism Linux uses to create new processes, including those started by the shell.

Activities:

- `ps`, `ps aux`, `ps -ef` — list currently running processes and their PIDs.

- top (or htop) — view a live, continuously updating list of processes and resource usage.
- Run a command with & to send it to the background; use jobs to list background jobs.
- fg %1 and bg %1 — bring a job to the foreground or resume it in the background.
- kill <PID> and kill -9 <PID> — terminate a process gracefully or forcefully; killall <name> to kill by name.
- Compile and run a small C program that calls fork() to observe parent/child behaviour.

Sample Program (fork_demo.c):

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    } else if (pid == 0) {
        printf("Child process: PID = %d, Parent PID = %d\n",
getpid(), getppid());
    } else {
        wait(NULL); // parent waits for child to finish
        printf("Parent process: PID = %d, Child PID = %d\n",
getpid(), pid);
    }
    return 0;
}
```

Lab Tasks:

20. List all running processes with ps aux and identify the PID of your own shell.
21. Start a long-running command (e.g. sleep 100 &), find its PID with ps, and terminate it with kill.
22. Open top, observe CPU and memory usage, and identify the top 3 processes by CPU usage.

23. Compile and run `fork_demo.c`; run it several times and explain why the order of parent/child output may vary.
24. Modify `fork_demo.c` to create two children instead of one and print each child's PID.

Viva / Review Questions:

- What is the difference between a process and a program?
 - What does `fork()` return in the parent versus the child process?
 - What is the difference between `kill` and `kill -9`?
-

Week 6

Topic:

System Monitoring

Objective:

- Monitor memory, disk, and overall system resource usage from the command line.
- Interpret common system statistics to identify potential bottlenecks.

Background / Theory:

System monitoring tools expose statistics that the kernel maintains about memory, storage, CPU, and I/O. Regularly checking these values helps diagnose performance issues such as low memory, full disks, or high CPU contention, which are core concerns in operating system design.

Activities:

- `free -h` — view total, used, and available RAM and swap space in human-readable form.
- `df -h` — view disk space usage per mounted filesystem.
- `du -sh dirname` — view the total size of a directory; `du -h --max-depth=1` for a per-subfolder breakdown.
- `vmstat 1 5` — sample virtual memory, process, and CPU statistics every second for 5 iterations.
- `uptime` — check system load averages and how long the system has been running.
- `iostat / netstat` (if available) — inspect disk I/O and network statistics.

Lab Tasks:

25. Record your system's total and available memory using `free -h`.
26. Use `df -h` to report how much space is used and free on your root filesystem.
27. Use `du -sh` on your home directory and identify the two largest subdirectories.
28. Run `vmstat 1 5` and explain what the 'free', 'si/so', and 'us/sy/id' columns represent.
29. Compare load average from uptime with the number of CPU cores on your system (`nproc`) and comment on whether the system is under heavy load.

Viva / Review Questions:

- What is the difference between used memory and available memory?
 - What does a high value in the 'wa' (I/O wait) column of `vmstat` indicate?
 - How is swap space used when physical RAM is exhausted?
-

Week 7

Topic:

Mid Lab Test

Objective:

- Demonstrate proficiency in Linux fundamentals, process management, and system monitoring through a practical examination.

Background / Theory:

This session is a supervised practical examination. It draws on all topics covered in Weeks 1-6: navigation and file management, permissions and shell scripting, process management, and system monitoring.

Activities:

- Review all commands and concepts from Weeks 1-6 before the test.
- Attempt the practical examination individually within the allotted time.
- Submit command outputs, scripts, and/or screenshots as required by the instructor.

Lab Tasks:

30. Complete the practical tasks assigned by the instructor for the mid-lab examination.
31. Ensure all requested outputs (files, scripts, screenshots, terminal transcripts) are saved and submitted before the deadline.

32. Be ready to briefly explain your approach for each task if asked.

Viva / Review Questions:

- Questions will be drawn from Weeks 1-6: Linux basics, file/permission management, shell scripting, process management, and system monitoring.
-

Week 8

Topic:

CPU Scheduling — FCFS and SJF

Objective:

- Understand the goals of CPU scheduling and key performance metrics.
- Implement and trace First-Come-First-Served (FCFS) and Shortest Job First (SJF) scheduling.

Background / Theory:

CPU scheduling decides which ready process runs on the CPU next. Two common metrics are Waiting Time (time a process spends ready but not running) and Turnaround Time (total time from arrival to completion) = Completion Time – Arrival Time; Waiting Time = Turnaround Time – Burst Time.

FCFS (First-Come-First-Served) is non-preemptive: processes run strictly in order of arrival. It is simple but can cause the 'convoy effect', where short processes wait behind one long process.

SJF (Shortest Job First), in its non-preemptive form, always picks the ready process with the smallest burst time next. It minimizes average waiting time but requires knowing burst times in advance and can starve longer jobs.

Activities:

- Manually trace a Gantt chart for a given set of processes using FCFS.
- Manually trace a Gantt chart for the same processes using non-preemptive SJF.
- Implement FCFS and SJF in C, compute waiting and turnaround times, and compare the averages.

Sample Program (fcfs.c):

```
#include <stdio.h>
```

```

int main() {
    int n, bt[20], wt[20], tat[20], at[20];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter arrival and burst time of each
process:\n");
    for (int i = 0; i < n; i++)
        scanf("%d %d", &at[i], &bt[i]);

    // Assumes processes are given already sorted by arrival
time (FCFS order)
    wt[0] = 0;
    for (int i = 1; i < n; i++)
        wt[i] = wt[i-1] + bt[i-1];

    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        totalWT += wt[i];
        totalTAT += tat[i];
        printf("P%d\tWT=%d\tTAT=%d\n", i+1, wt[i], tat[i]);
    }
    printf("Average Waiting Time = %.2f\n", totalWT / n);
    printf("Average Turnaround Time = %.2f\n", totalTAT /
n);
    return 0;
}

```

Lab Tasks:

33. Given: P1(AT=0,BT=5), P2(AT=1,BT=3), P3(AT=2,BT=8), P4(AT=3,BT=6) — draw the Gantt chart and compute WT/TAT for FCFS by hand.
34. Repeat the same process set for non-preemptive SJF and compare average waiting times against FCFS.
35. Compile and run fcfs.c with the given input and verify your hand-calculated results.
36. Extend the logic (on paper or in code) to implement non-preemptive SJF by sorting processes by burst time before computing wt[] and tat[].

37. Write a short comparison: under what conditions does SJF clearly outperform FCFS?

Viva / Review Questions:

- Why can FCFS lead to poor average waiting time (the convoy effect)?
 - Why is non-preemptive SJF called 'optimal' for minimizing average waiting time, and what is its main drawback?
 - What information does the scheduler need in advance to run SJF, and why is this often unrealistic?
-

Week 9

Topic:

Preemptive/Non-preemptive Scheduling — RR, Priority, SRTF

Objective:

- Understand preemption and implement Round Robin, Priority, and Shortest Remaining Time First scheduling.
- Compare preemptive and non-preemptive scheduling behaviour and performance.

Background / Theory:

Round Robin (RR) is preemptive: each process gets a fixed time quantum; if it doesn't finish, it is placed at the back of the ready queue and the next process runs. RR is fair and responsive but performance depends heavily on the chosen quantum size.

Priority scheduling assigns each process a priority number; the highest-priority ready process runs next. It can be preemptive (a higher-priority arrival interrupts the running process) or non-preemptive, and can suffer starvation of low-priority processes unless aging is used.

SRTF (Shortest Remaining Time First) is the preemptive version of SJF: if a new process arrives with a shorter remaining burst time than the currently running process, the CPU switches to it.

Activities:

- Manually trace a Gantt chart for a given process set using Round Robin with a specified time quantum.

- Manually trace the same set using Priority scheduling (both preemptive and non-preemptive).
- Manually trace the same set using SRTF and compare with SJF from Week 8.
- Implement Round Robin scheduling in C using a circular ready queue.

Sample Program (round_robin.c):

```
#include <stdio.h>

int main() {
    int n, bt[20], rt[20], wt[20], tat[20], quantum;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time of each process:\n");
    for (int i = 0; i < n; i++) { scanf("%d", &bt[i]); rt[i]
= bt[i]; }
    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    int t = 0, done;
    do {
        done = 1;
        for (int i = 0; i < n; i++) {
            if (rt[i] > 0) {
                done = 0;
                int slice = (rt[i] > quantum) ? quantum :
rt[i];

                t += slice;
                rt[i] -= slice;
                if (rt[i] == 0) tat[i] = t;
            }
        }
    } while (!done);

    float totalWT = 0, totalTAT = 0;
    for (int i = 0; i < n; i++) {
        wt[i] = tat[i] - bt[i];
        totalWT += wt[i];
        totalTAT += tat[i];
    }
}
```

```

        printf("P%d\tTAT=%d\tWT=%d\n", i+1, tat[i], wt[i]);
    }
    printf("Average Waiting Time = %.2f\n", totalWT / n);
    printf("Average Turnaround Time = %.2f\n", totalTAT /
n);
    return 0;
}

```

Lab Tasks:

38. Using P1(BT=10), P2(BT=5), P3(BT=8) and quantum=3, draw the Gantt chart for Round Robin by hand, then verify with round_robin.c.
39. Assign priorities to the same three processes and trace both preemptive and non-preemptive Priority scheduling.
40. Trace SRTF for a process set with staggered arrival times and compare the result to SJF from Week 8.
41. Run round_robin.c with quantum values of 1, 3, and 6; record how average waiting time changes and explain the trend.
42. Summarize, in your own words, when RR is preferable to FCFS/SJF and when Priority scheduling risks starvation.

Viva / Review Questions:

- What happens to average waiting time as the Round Robin time quantum becomes very large or very small?
- How does aging help prevent starvation in Priority scheduling?
- What triggers a context switch in SRTF that would not occur in non-preemptive SJF?

Week 10

Topic:

Memory Management — First Fit, Best Fit, Worst Fit

Objective:

- Understand contiguous memory allocation strategies used by the OS.
- Implement and compare First Fit, Best Fit, and Worst Fit allocation algorithms.

Background / Theory:

In contiguous memory allocation, the OS keeps a list of free memory blocks (holes) and must decide which hole to use when a new process requests memory.

First Fit allocates the first hole that is large enough, scanning from the start of memory. It is fast but can leave many small unusable fragments near the start.

Best Fit searches all holes and picks the smallest one that is still large enough, minimizing leftover space per allocation but requiring a full scan and often leaving many tiny, unusable fragments overall.

Worst Fit picks the largest available hole, leaving a bigger leftover fragment that may be more likely to be useful for future requests, though it tends to waste large blocks quickly.

Activities:

- Given a set of memory block sizes and process sizes, manually allocate processes using First Fit, Best Fit, and Worst Fit.
- Implement First Fit allocation in C, then adapt it for Best Fit and Worst Fit.
- Compare the resulting fragmentation and unused memory for each strategy.

Sample Program (first_fit.c):

```
#include <stdio.h>

int main() {
    int m, n;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int block[20], blockCopy[20];
    for (int i = 0; i < m; i++) scanf("%d", &block[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int proc[20], allocation[20];
    for (int i = 0; i < n; i++) { scanf("%d", &proc[i]);
allocation[i] = -1; }
    for (int i = 0; i < m; i++) blockCopy[i] = block[i];

    // First Fit
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockCopy[j] >= proc[i]) {
```

```

        allocation[i] = j;
        blockCopy[j] -= proc[i];
        break;
    }
}

printf("\nProcess No.\tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t\t%d\t\t", i+1, proc[i]);
    if (allocation[i] != -1) printf("%d\n",
allocation[i] + 1);
    else printf("Not Allocated\n");
}
return 0;
}

```

Lab Tasks:

43. Given memory blocks {100, 500, 200, 300, 600} and processes {212, 417, 112, 426}, manually allocate using First Fit, Best Fit, and Worst Fit, and record which process goes to which block.
44. Compile and run `first_fit.c` with the same input and verify your manual First Fit results.
45. Modify the program's inner loop to implement Best Fit (choose the smallest suitable block) and re-run it on the same input.
46. Modify the program's inner loop to implement Worst Fit (choose the largest suitable block) and re-run it on the same input.
47. Tabulate the leftover memory in each block for all three strategies and comment on which strategy wasted the least memory overall for this input.

Viva / Review Questions:

- Why might Best Fit still lead to a lot of wasted memory over time, despite minimizing leftover space per allocation?
 - What is the time complexity of First Fit versus Best Fit/Worst Fit, and why?
 - Give an example where Worst Fit performs better than First Fit for a future allocation.
-

Week 11

Topic:

Memory Management — Fragmentation and Compaction

Objective:

- Distinguish between internal and external fragmentation.
- Understand and simulate compaction as a technique to reduce external fragmentation.

Background / Theory:

Internal fragmentation occurs when a process is allocated a memory block larger than it needs, wasting the unused space within that block (common with fixed-size partitioning or paging).

External fragmentation occurs when free memory is broken into small, scattered holes that are individually too small to satisfy a request, even though the total free memory would be sufficient (common with First/Best/Worst Fit over time).

Compaction reduces external fragmentation by relocating allocated processes so that all free memory is combined into one contiguous block. It requires that processes be relocatable (using relocation registers/base addresses) and has a runtime cost proportional to the amount of memory moved.

Activities:

- Given a memory layout with allocated and free blocks scattered throughout, calculate the total free memory and identify whether a given process request can be satisfied without compaction.
- Manually perform compaction on a fragmented memory layout and recompute the largest available contiguous block afterward.
- Extend the Week 10 allocation program to print total free memory and the largest free block before and after a simulated compaction step.

Lab Tasks:

48. Given memory blocks in the order [Free 100K][P1 200K][Free 60K][P2 150K][Free 90K], calculate total free memory and the largest usable free block before compaction.
49. Perform compaction by hand: move P1 and P2 to be contiguous from the start of memory, and show the new memory layout with one combined free block.

50. Using your Week 10 First Fit code (or block layout), simulate an allocation request that fails due to external fragmentation despite enough total free memory, then show it succeeding after compaction.
51. Discuss, based on your simulation, one practical drawback of compaction (e.g. overhead of moving processes, need for relocatable code).
52. Give one real example each of internal fragmentation and external fragmentation you observed or can construct from this week's exercises.

Viva / Review Questions:

- Why doesn't compaction help with internal fragmentation?
 - What system feature must be present for the OS to safely relocate a running process's memory during compaction?
 - Why is compaction expensive, and when might an OS choose to avoid it (e.g. by using paging instead)?
-

Week 12

Topic:

Page Replacement — FIFO, LRU, Optimal

Objective:

- Understand paging and why page replacement algorithms are needed.
- Implement and compare FIFO, LRU, and Optimal page replacement, and count page faults.

Background / Theory:

In a paged, demand-paging virtual memory system, when a requested page is not in memory (a page fault) and memory is full, the OS must evict an existing page to make room. Different algorithms choose different victims.

FIFO (First-In-First-Out) evicts the page that has been in memory the longest, regardless of how recently it was used. It is simple to implement (a queue) but can suffer from Belady's Anomaly, where increasing the number of frames increases the number of page faults.

LRU (Least Recently Used) evicts the page that has not been used for the longest time, approximating future behaviour using the past. It performs well in practice but needs extra bookkeeping (timestamps or a stack) to track usage order.

Optimal replacement evicts the page that will not be used for the longest time in the future. It gives the minimum possible number of page faults for a given reference string but requires future knowledge, so it is used only as a theoretical benchmark.

Activities:

- Manually trace FIFO, LRU, and Optimal page replacement for a given reference string and frame count, marking hits and faults.
- Implement FIFO page replacement in C and count total page faults.
- Extend or adapt the program to implement LRU, then compare fault counts across all three algorithms for the same input.

Sample Program (fifo_paging.c):

```
#include <stdio.h>

int main() {
    int pages[20], n, frameCount, frames[10], faults = 0,
    index = 0;
    printf("Enter number of pages in reference string: ");
    scanf("%d", &n);
    printf("Enter reference string: ");
    for (int i = 0; i < n; i++) scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &frameCount);
    for (int i = 0; i < frameCount; i++) frames[i] = -1;

    for (int i = 0; i < n; i++) {
        int found = 0;
        for (int j = 0; j < frameCount; j++)
            if (frames[j] == pages[i]) { found = 1; break; }

        if (!found) {
            frames[index] = pages[i];        // FIFO
replacement
            index = (index + 1) % frameCount;
            faults++;
        }
        printf("Ref %d -> Frames: ", pages[i]);
        for (int j = 0; j < frameCount; j++) printf("%d ",
```

```
frames[j]);
    printf("%s\n", found ? "(hit)" : "(fault)");
}
printf("Total Page Faults = %d\n", faults);
return 0;
}
```

Lab Tasks:

53. For reference string 7,0,1,2,0,3,0,4,2,3,0,3,2 with 3 frames, trace FIFO by hand and count page faults.
54. Repeat the same reference string and frame count for LRU, then for Optimal, and compare total faults across all three.
55. Compile and run `fifo_paging.c` with the same input and verify your hand-traced FIFO result.
56. Increase the frame count to 4 and re-run FIFO; check whether the number of page faults increases or decreases, and relate this to Belady's Anomaly.
57. Adapt `fifo_paging.c`'s replacement rule to implement LRU (replace the page unused for the longest time) and report the new fault count for the original reference string.

Viva / Review Questions:

- What is Belady's Anomaly and which algorithm(s) can exhibit it?
 - Why is LRU generally preferred over FIFO in real operating systems despite the extra bookkeeping cost?
 - Why can't a real OS implement Optimal replacement directly?
-

Week 13

Topic:

Performance Evaluation & Lab Final

Objective:

- Consolidate and compare the scheduling, memory management, and page replacement algorithms studied throughout the course.
- Demonstrate overall understanding through a final practical viva.

Background / Theory:

This session brings together the CPU scheduling algorithms (Weeks 8-9), memory allocation strategies (Weeks 10-11), and page replacement algorithms (Week 12) studied earlier, comparing them on standard metrics: average waiting/turnaround time for scheduling, memory utilization/fragmentation for allocation, and page fault count for replacement.

Activities:

- Re-run your FCFS, SJF, Round Robin, and Priority scheduling programs on one common process set and tabulate average waiting/turnaround times side by side.
- Re-run your First Fit, Best Fit, and Worst Fit programs on one common memory/process set and tabulate leftover memory for each.
- Re-run your FIFO, LRU, and Optimal paging programs on one common reference string and tabulate page faults for each.
- Prepare a short summary table and be ready to explain the trends to the instructor during the viva.

Lab Tasks:

58. Produce one comparison table for scheduling algorithms (FCFS, SJF, RR, Priority) showing average waiting time and turnaround time for the same process set.
59. Produce one comparison table for memory allocation (First Fit, Best Fit, Worst Fit) showing allocation success/failure and leftover memory per block.
60. Produce one comparison table for page replacement (FIFO, LRU, Optimal) showing total page faults for the same reference string and frame count.
61. Write a one-paragraph conclusion for each of the three areas, stating which algorithm performed best for your test data and why.
62. Be prepared to answer viva questions covering any lab performed during the semester.

Viva / Review Questions:

- Across your results, which scheduling algorithm gave the lowest average waiting time, and does this match the theoretical expectation?
 - Which memory allocation strategy left the least wasted memory in your test case, and is this generally true?
 - How close did LRU's page fault count come to Optimal in your trace, and why is that gap expected?
-

Week 14

Topic:

Advanced Topic — Deadlock Handling (Banker's Algorithm)

Objective:

- Understand the four necessary conditions for deadlock.
- Understand and manually apply the Banker's Algorithm for deadlock avoidance.
- (This session is reserved/flexible — the instructor may substitute an alternative advanced topic or a mini-project in its place.)

Background / Theory:

A deadlock is a state where a set of processes are each waiting for a resource held by another process in the set, so none can proceed. Deadlock requires four conditions to hold simultaneously: mutual exclusion, hold-and-wait, no preemption, and circular wait.

The Banker's Algorithm is a deadlock-avoidance algorithm. Given each process's maximum resource need, current allocation, and the system's available resources, it checks whether granting a request leaves the system in a 'safe state' — i.e. there exists some order in which all processes can still finish — before actually granting the request.

Activities:

- Work through a Banker's Algorithm example by hand: compute the Need matrix ($\text{Max} - \text{Allocation}$) and find a safe sequence given Available resources.
- Simulate a resource request and use the safety algorithm to decide whether it can be granted immediately.
- (If an alternate topic is assigned instead, replace this activity with the instructor's specified exercise, e.g. a mini-project demo or additional review.)

Lab Tasks:

63. Given Allocation, Max, and Available matrices for 5 processes and 3 resource types, compute the Need matrix.
64. Apply the safety algorithm to find a safe sequence, or show that none exists.
65. Given an additional resource request from one process, determine whether it can be granted safely and justify your answer using the algorithm.

66.If your instructor has assigned a different topic or mini-project for this session, complete and document that task instead, following the same lab-report format used in previous weeks.

Viva / Review Questions:

- What are the four necessary conditions for deadlock, and how does removing any one of them prevent deadlock?
- What is a 'safe state' in the Banker's Algorithm, and how does it differ from a deadlocked state?
- What information must the OS know in advance for the Banker's Algorithm to work, and why can this be a limitation in practice?

Assessment

Component	Marks
Attendance	10
Lab Performance(Report)	15
Lab Assessment (pre-mid)	10
Mid	10
Final (with Viva)	30
Quiz	25

Total: 100 marks